

Documentation Technique Ultra Complete - Application Lourde Epreuve E6

Version du document: 3.0 Date: 20/04/2026 Public: Jury technique et non technique, techniciens, maintenance applicative

Comment lire ce document

Ce document est volontairement structure en 3 niveaux:

1. Niveau A - Lecture metier (non developpeur)
2. Niveau B - Lecture technicienne (exploitation, support, procedure)
3. Niveau C - Lecture developpeur (architecture, code, SQL, evolution)

Objectif: permettre a un membre du jury non dev de comprendre le projet, tout en donnant un niveau de detail suffisant a un jury dev/technique.

Niveau A - Vue claire pour non developpeur

A1. Resume executif

L'application est un logiciel desktop de gestion interne pour un drive alimentaire. Elle permet:

- de se connecter avec des comptes internes
- de gerer le stock de produits
- de preparer les commandes clients
- de gerer les promotions commerciales
- d'administrer les utilisateurs internes et les clients web

Valeur metier:

- meilleure fiabilite de preparation des commandes
- meilleur suivi des stocks
- reduction des erreurs manuelles
- outil unique pour administration et exploitation

A2. Fonctions principales (langage metier)

Connexion

L'utilisateur s'authentifie avec email/mot de passe.

Stock

L'operateur consulte les produits, cherche un article, modifie un produit ou ajoute un nouveau produit.

Commandes

Le preparateur voit les commandes en attente, valide les lignes preparees, puis valide la commande complete.

Promotions

Le service marketing/edition cree, modifie et supprime des promotions.

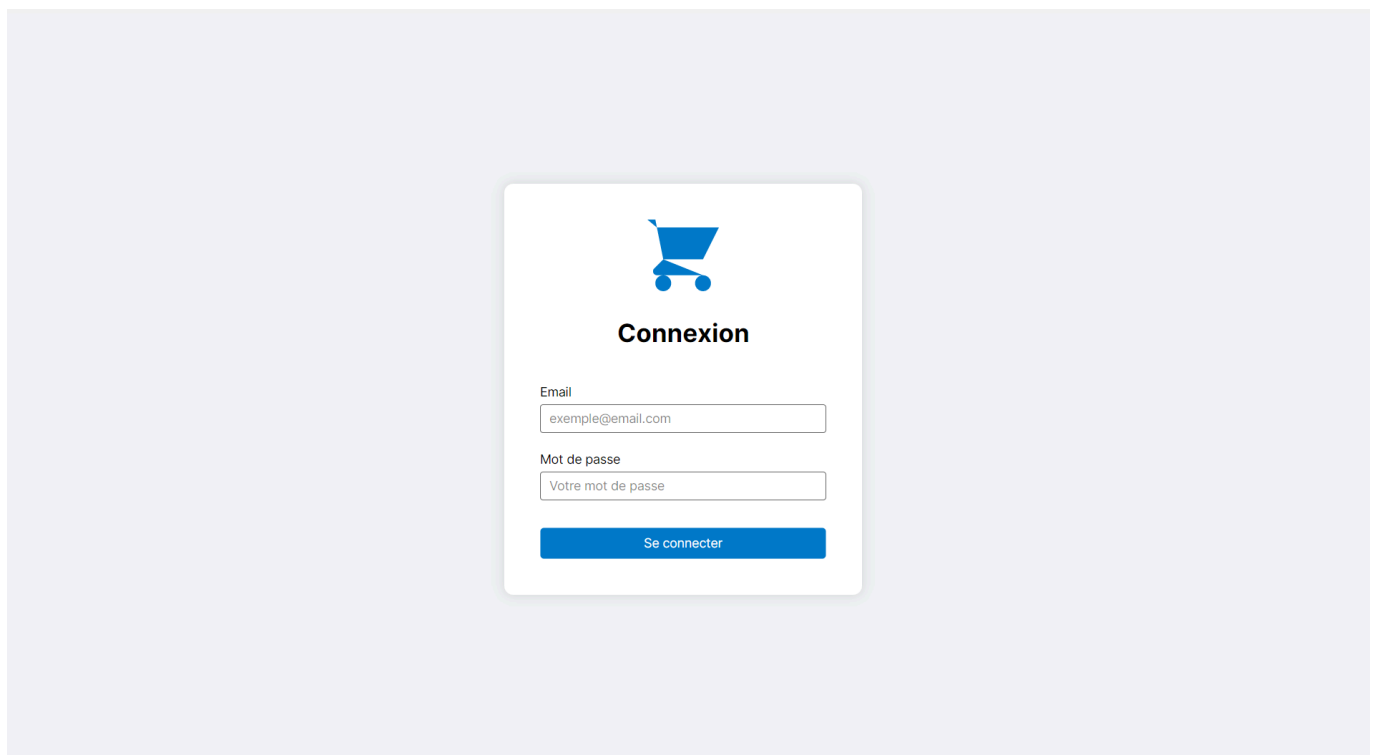
Administration

L'administrateur gere:

- les comptes des utilisateurs de l'application
- les clients venant du site web

A3. Ecrans (preuves visuelles)

Ecran connexion



Ecran stock

Stock

Commandes

Promotions

Admin

Déconnexion

Gestion du Stock

Consultez et modifiez vos produits

Rechercher un produit...

Référence	Libellé	Catégorie	Prix HT	Stock	Ima	Actions
FRL001	Pommes Bio (1kg)	Fruits & Légumes	3,99 €	150	☒	Modi
FRL002	Tomates en Grappe (1kg)	Fruits & Légumes	2,99 €	80	☒	Modi
FRL003	Bananes (1kg)	Fruits & Légumes	1,89 €	200	☒	Modi
FRL004	Carottes des Sables (1kg)	Fruits & Légumes	1,50 €	100	☒	Modi
BOU001	Baguette Tradition	Boulangerie	1,10 €	120	☐	Modi
BOU002	Pain Complet	Boulangerie	2,50 €	45	☐	Modi
BOU003	Croissants au Beurre (x4)	Boulangerie	3,80 €	30	☐	Modi
CRE001	Lait Bio Demi-Écrémé (1L)	Crèmerie	1,45 €	90	☐	Modi
CRE002	Yaourts Nature (x8)	Crèmerie	3,20 €	60	☐	Modi
CRE003	Beurre Doux (250g)	Crèmerie	2,80 €	75	☐	Modi

Ajouter un produit

Actualiser

Ecran commandes

Stock

Commandes

Promotions

Admin

Déconnexion

Commandes à Préparer

Numéro	Retrait	Total
CMD-2024-001	26/03/2026 09:48:49	27,43 €
CMD-2024-005	26/03/2026 11:48:49	34,32 €

Détails de la commande

Produit	Qté	Statut	Action
Pommes Bio (1kg)	5	A_VALIDER	Valider
Pain Complet	2	A_VALIDER	Valider
Carottes des Sables (1kg)	3	A_VALIDER	Valider

Valider la commande et envoyer l'email

Ecran promotions

Gestion Epreuve E6

Stock

Commandes

Promotions

Admin

Déconnexion

Gestion des Promotions

Nouvelle Promotion

ID	Titre	Badge	Prix	Actions
1	Sélection Café	-30%	Dès 4,99 €	ModifierSupprimer
2	Légumes de saison	2+1 OFFERT	2,50 € / kg	ModifierSupprimer
3	Rayon Bio	NOUVEAU	Dès 1,99 €	ModifierSupprimer

Ecran administration utilisateurs appli

Gestion Epreuve E6

Stock

Commandes

Promotions

Admin

Déconnexion

Tableau de bord Administrateur

Utilisateurs Appli Lourde

Utilisateurs Web (Clients)

Gestion des comptes employés/admins

Ajouter un utilisateur

ID	Nom	Email	Admin	Rôle	Créé le	Action
1	Administrateur	admin@drive.test	☒	admin	26/03/2026 07:48:48	Modifier
2	Editeur App	editeur@drive.test	☐	editeur	26/03/2026 07:48:49	Modifier
3	Jean Dupont	user@drive.test	☐	client	26/03/2026 07:48:49	Modifier

Ecran administration clients web

Gestion Epreuve E6

Stock

Commandes

Promotions

Admin

Déconnexion

Tableau de bord Administrateur

Utilisateurs Appli Lourde

Utilisateurs Web (Clients)

Liste des clients inscrits sur le site web

ID	Nom	Prénom	Email	Téléphone	Inscrit le	Action
1	Martin	Sophie	sophie.martin@example.com	0612345678	26/03/2026 07:48:45	Modifier
2	Durand	Pierre	pierre.durand@example.com	0698765432	26/03/2026 07:48:45	Modifier

A4. Roles utilisateurs

Role	Acces
admin	tous les modules
editeur	stock, commandes, promotions
utilisateur standard	stock, commandes

A5. Risques metier et reponses

Risque	Impact	Reponse actuelle
panne base de donnees	impossible de charger les donnees	diagnostic au demarrage + logs
erreur preparation commande	retard client	validation ligne par ligne
mot de passe vole	acces non autorise	hash BCrypt
echec email	client non informe	trace d'erreur + continuation de l'app

Niveau B - Vue technicienne (exploitation/support)

B1. Fiche technique rapide

- Type: application desktop Avalonia
 - Runtime: .NET 10
 - Base de donnees: MariaDB
 - Data access: Dapper + MySQLConnector
 - Pattern: MVVM
 - Injection de dependances: Microsoft.Extensions.DependencyInjection
-

B2. Prerequis d'installation

- Windows 10/11
 - .NET SDK 10.0 ou superieur
 - MariaDB accessible
 - Base drive_db disponible
-

B3. Installation pas a pas

1. Cloner le depot
2. Importer le dump SQL dump-drive_db-202512051502.sql dans la base drive_db
3. Verifier appsettings.json
4. Restaurer les packages
5. Construire
6. Lancer

Commandes:

```
dotnet restore
dotnet build application_lourde_avalonia.csproj
dotnet run --project application_lourde_avalonia.csproj
```

Note importante:

Le dossier contient un .sln et un .csproj. Pour eviter les ambiguïtées, utiliser toujours --project.

B4. Configuration locale

Fichier: appsettings.json

Parametre cle:

- ConnectionStrings:DefaultConnection

Exemple local observe:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
    "Server=localhost;Port=3306;Database=drive_db;Uid=root;Pwd=;"
  }
}
```

B5. Demarrage et verification technique

Au lancement, l'application:

1. charge appsettings.json
2. teste la connexion DB
3. execute une migration technique inline
4. construit le conteneur DI
5. démarre l'UI

Traces console attendues:

- [DIAGNOSTIC] Tentative de connexion...
- [DIAGNOSTIC] CONNEXION REUSSIE
- [MIGRATION] ...

B6. Operations de support

Verifier statut de fonctionnement

Checklist:

- application se lance
- login fonctionne
- stock charge
- commandes chargees
- navigation ok

Nettoyage build


```
dotnet clean application_lourde_avalonia.csproj
```

Relance propre

```
dotnet run --project application_lourde_avalonia.csproj
```

B7. Guide de depannage (N1/N2)

Cas 1 - Erreur de connexion DB

Symptomes:

- [DIAGNOSTIC] ECHEC CONNEXION
- ecrans vides

Actions:

1. verifier service MariaDB
2. verifier host/port DB
3. verifier user/mot de passe DB
4. verifier nom de base drive_db

Cas 2 - L'app ne se lance pas avec dotnet run

Cause probable:

- ambiguite solution/projet

Action:

```
dotnet run --project application_lourde_avalonia.csproj
```

Cas 3 - Email non envoye

Actions:

1. verifier acces reseau SMTP
2. verifier identifiants SMTP
3. verifier blocage firewall/antispam

B8. Securite exploitation

Points deja presents

- mots de passe users verifies en BCrypt
- requetes SQL parametrees via Dapper

Point critique a corriger

Les identifiants SMTP sont actuellement dans le code (EmailService). En contexte reel, cela doit etre externalise (variables d'environnement/secrets manager).

B9. Sauvegarde et reprise

Sauvegarde recommandee

- dump MariaDB quotidien
- retention minimum 7 jours

Procedure de reprise

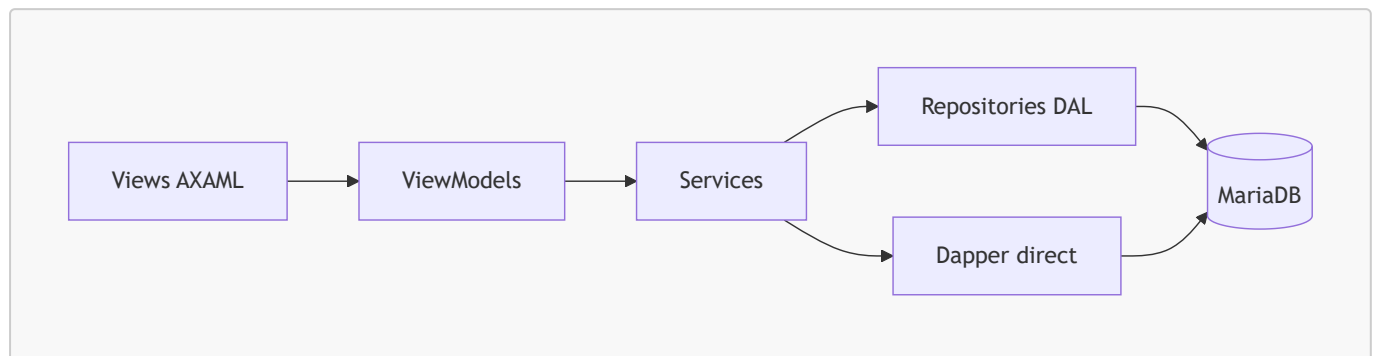
1. restaurer le dump DB
 2. verifier appsettings
 3. relancer l'app
 4. faire tests smoke (login, stock, commandes)
-

B10. Tests de non regression (technicien)

- login valide
- login invalide
- recherche produit
- ajout produit
- edition produit
- validation ligne commande
- validation commande
- ajout promotion
- suppression promotion
- edition utilisateur admin
- edition client web

Niveau C - Vue developpeur / jury technique

C1. Architecture logique



C2. Structure du code

- Assets/: ressources graphiques
- DAL/: interfaces + repositories SQL
- Infrastructure/: extension de cablage DI
- Models/: objets metier
- Services/: logique metier
- ViewModels/: logique presentation MVVM
- Views/: ecrans AXAML
- Program.cs: bootstrap

C3. Bootstrap detaille

Fichier: Program.cs

Sequence technique:

1. ConfigurationBuilder + AddJsonFile
2. lecture connection string
3. test MySqlConnection.Open()
4. migration inline:
 - conversion image vers LONGTEXT
 - tentative ajout colonnes promotions (try/catch)
5. AddFrameworkAdapters(configuration)
6. BuildServiceProvider
7. App.Services = serviceProvider
8. BuildAvaloniaApp().StartWithClassicDesktopLifetime(args)

C4. Dependency Injection detaillee

DAL

- IDbConnector -> MariaDbConnector
- IProduitRepository -> ProduitRepository
- ICommandeRepository -> CommandeRepository

Services

- IAuthService -> AuthService
- IStockService -> StockService
- ICommandeService -> CommandeService
- IAdminService -> AdminService
- IPromotionService -> PromotionService
- IEmailService -> EmailService
- IDialogService -> DialogService

ViewModels

- MainViewModel
- LoginViewModel
- StockViewModel
- CommandeViewModel
- AdminDashboardViewModel
- PromotionsViewModel

C5. Description composants

C5.1 MainViewModel

Responsabilites:

- gestion session locale (CurrentUser, IsLoggedIn, IsAdmin, IsEditor)
- navigation entre modules
- role-based visibility

Commandes:

- ShowStock
- ShowCommandes
- ShowPromotions
- ShowAdminDashboard
- Logout

C5.2 LoginViewModel + AuthService

Flux:

1. validation champs non vides
 2. appel LoginAsync
 3. Query user by email
 4. BCrypt.Verify
 5. callback OnLoginSuccess
-

C5.3 StockViewModel + ProduitRepository

Flux:

- LoadProduits -> SELECT produits
- SearchText -> filtrage en memoire (reference/libelle)
- AddProduit/EditProduit -> INSERT/UPDATE
- SaveStock -> UPDATE stock

Gestion image:

- conversion binaire en data URI base64 dans EditProduitViewModel
-

C5.4 CommandeViewModel + CommandeService + CommandeRepository

Flux:

- LoadCommandes -> WHERE statut = A_PREPARER
 - selection commande -> load lignes
 - ValidateLigne -> statut_ligne PREPAREE
 - ValidateCommande -> statut commande PREPAREE + email client
-

C5.5 PromotionsViewModel + PromotionService

Flux:

- LoadPromotions -> SELECT promotions
- Add/Update/Delete promotion
- liaison produit possible via ProduitId (schema cible)

C5.6 AdminDashboardViewModel + AdminService

Flux:

- chargement users et clients
 - AddUser -> hash mot de passe BCrypt
 - UpdateUser / UpdateClient
-

C6. Schema base de donnees

C6.1 Tables metier effectives

- users
- clients
- produits
- promotions
- commandes
- lignes_commandes

C6.2 Colonnes cles

users

- id, name, email, password, is_admin, role, created_at, updated_at

clients

- id, nom, prenom, email, telephone, created_at, updated_at

produits

- id, reference (unique), libelle, description, image (longtext), categorie, prix_ht, tva, stock, actif, created_at, updated_at

promotions (dump)

- id, titre, description, image_path, badge_text, badge_color, prix_affichage, texte_bouton, lien_url, created_at, updated_at

promotions (schema cible cote app)

- ◦ produit_id
- ◦ type_promo
- ◦ valeur_promo
- ◦ min_quantite

commandes

- id, client_id, numero_commande, statut, creneau_retrait, total_ht, total_ttc, note_interne, created_at, updated_at

lignes_commandes

- id, commande_id, produit_id, quantite_demandee, quantite_preparee, prix_unitaire_ht, statut_ligne, note

C6.3 Relations

- commandes.client_id -> clients.id (SET NULL)
- lignes_commandes.commande_id -> commandes.id (CASCADE)
- lignes_commandes.produit_id -> produits.id

C7. Contrats interfaces

Services

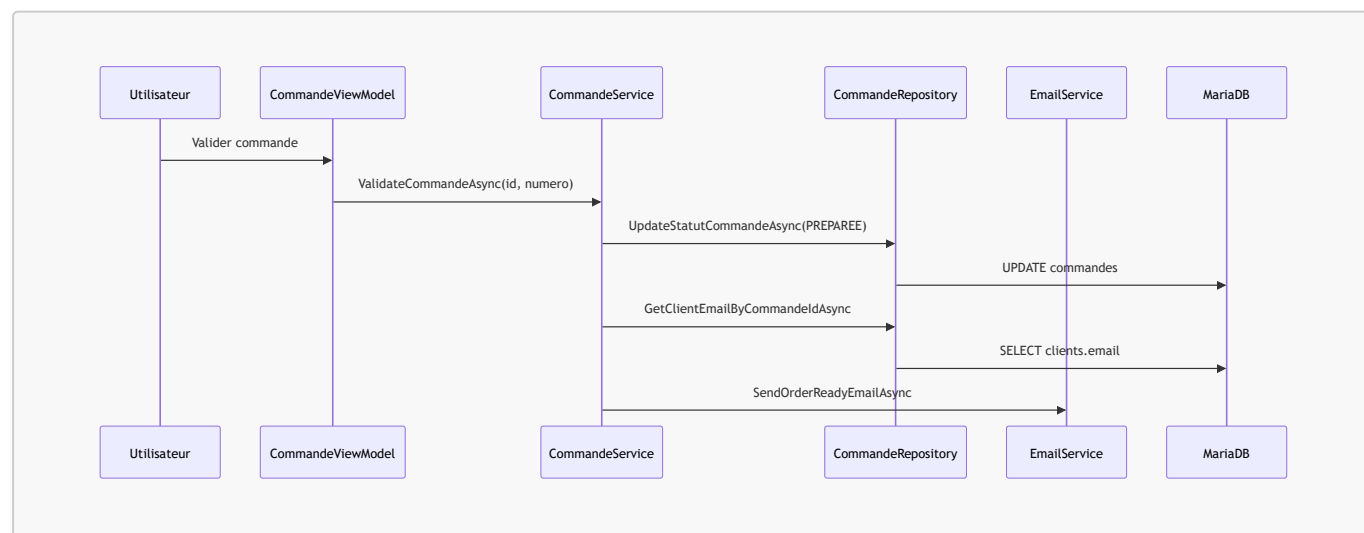
- IAuthService
- IStockService
- ICommandeService
- IAdminService
- IPromotionService
- IEmailService
- IDialogService

DAL

- IDbConnector
- IProduitRepository
- ICommandeRepository

C8. Flux metier detaille

C8.1 Flux "Validation commande"



C8.2 Flux "Ajout produit"

1. ouverture modal EditProduitWindow
 2. saisie informations + image optionnelle
 3. Save -> StockService.AddProduitAsync
 4. ProduitRepository.AddAsync -> INSERT produits
 5. refresh liste
-

C9. Securite technique

C9.1 Bonnes pratiques en place

- hash BCrypt
- SQL parametre
- separation des couches

C9.2 Points faibles

- credentials SMTP en dur
- migration DB inline non versionnee
- absence de couche d'audit applicatif

C9.3 Plan de remediations priorise

1. externalisation secrets
 2. migration outillee (versionning)
 3. logs structures centralises
 4. durcissement autorisations fines
-

C10. Qualite et tests

C10.1 Etat actuel

- verification manuelle fonctionnelle
- pas de suite de tests automatisee committee

C10.2 Plan recommande

- tests unitaires services
- tests integration DAL (DB de test)
- tests UI smoke
- pipeline CI: build + tests + artefact

C11. Performance et scalabilite

C11.1 Observations

- DataGrid charges en memoire complete
- filtrage local
- pas de pagination SQL

C11.2 Optimisations proposees

- pagination cote SQL
- indexation colonnes critiques
- lazy loading sur ecrans volumineux

C12. Journalisation et observabilite

C12.1 Traces existantes

- Console.WriteLine boot diagnostics
- Debug.WriteLine erreurs metier

C12.2 Cible recommandee

- logs structures avec niveaux
- correlation id operation
- journal audit actions admin

C13. Matrice exigences -> composants -> preuves

Exigence	Composant principal	Preuve
login securise	LoginViewModel + AuthService	BCrypt.Verify + messages d'erreur
gestion stock	StockViewModel + ProduitRepository	ecran stock + CRUD
preparation commandes	CommandeViewModel + CommandeService	ecran commandes + validation
envoi email commande	CommandeService + EmailService	appel SendOrderReadyEmailAsync
gestion promotions	PromotionsViewModel + PromotionService	ecran promotions
administration comptes	AdminDashboardViewModel + AdminService	ecrans admin users/clients

C14. Procedure de recette jury (simple)

1. Lancer application
2. Se connecter avec compte admin
3. Ouvrir module stock et faire une recherche
4. Ouvrir module commandes et valider une ligne
5. Ouvrir module promotions et ajouter une promotion test
6. Ouvrir module admin et modifier un utilisateur
7. Se deconnecter

Critere de succes:

- aucune erreur bloquante
 - donnees rechargees correctement
 - navigation role-based conforme
-

C15. Conclusion generale

L'application est bien structuree pour un contexte de projet BTS SIO:

- architecture propre et lisible
- separation claire des responsabilites
- fonctionnalites metier couvertes
- base exploitable par un technicien

Pour un niveau production entreprise, les priorites sont:

- gestion des secrets
 - migration DB versionnee
 - tests automatiques
 - observabilite centralisee
-

Glossaire simplifie (pour jury non dev)

- MVVM: facon d'organiser le code UI pour separer affichage et logique
- DI (injection de dependances): mecanisme qui branche automatiquement les composants
- Dapper: outil leger pour executer du SQL depuis .NET
- BCrypt: methode de securisation des mots de passe
- CRUD: creer, lire, modifier, supprimer
- Smoke test: test rapide de bon fonctionnement global